NOISY REWARDS WRITE NOISY MEMORIES: REWARD ERROR AS A VARIANCE AMPLIFIER IN SELF-IMPROVING AGENTS

Anonymous authors
Paper under double-blind review

ABSTRACT

Self-improving agents often use task reward to decide how an experience should be written into persistent memory. This creates a direct path from reward error to future behavior. We study that path in the released ReasoningBank implementation used by a recent agent-fragility study. The code sends a trajectory with score one to a success-reflection writer. Score zero sends the identical trajectory to a failure-reflection writer. We intervene on this single reward bit while holding the released trajectory fixed. Four complete paired interventions all produce different memory content. The memory-item title set changes in every complete pair. Mean token-set Jaccard distance between paired memories is 0.735, with values from 0.691 to 0.770. The effect appears for trajectories that originally succeeded and for trajectories that originally failed. These results establish reward-conditioned memory writing as a concrete state channel: a reward-label perturbation survives the episode boundary as a different persistent memory. We then define a downstream variance experiment that injects paired memories into matched future tasks under fixed benchmark state. The protocol measures whether reward-induced memory divergence produces action divergence and success variance across repeated runs. This mechanism connects two empirical findings that have previously been studied separately: run variance in self-improving agents and reward inflation under persistent memory reuse. Reward quality is therefore part of the state-estimation problem for self-improving systems. A noisy reward signal can become a durable memory intervention before any parameter update occurs.

1 INTRODUCTION

Self-improving agents convert interaction histories into persistent state. External memory is a common implementation because it can update quickly without modifying model parameters. ReasoningBank writes reasoning memories from completed trajectories (Ouyang et al., 2026). Agent Workflow Memory stores reusable procedures from prior interactions (Wang et al., 2024). These systems make experience reusable across future tasks.

The memory writer needs a learning signal. In the released ReasoningBank implementation used by the recent fragility study (Ye et al., 2026), the terminal score controls which reflection objective writes the memory. Score one activates a success-reflection prompt. Score zero activates a failure-reflection prompt. The success writer asks what made the trajectory work. The failure writer asks what caused the attempt to fail. A one-bit reward change therefore changes the interpretation assigned to the same action history.

This implementation creates a persistent reward channel. A proxy evaluator can make an error at the end of an episode. The error then selects the wrong memory-writing objective. The resulting text is stored and becomes available to later tasks. Reward noise has crossed the episode boundary and entered the agent state.

The same fragility study reports substantial variance across repeated self-improvement runs. Its main experiments use ground-truth reward specifically to remove reward noise from the analysis (Ye et al., 2026). This design choice exposes an open causal question: how much of self-improvement variance can originate from reward-conditioned memory writing itself? Memory Reward Inflation supplies a

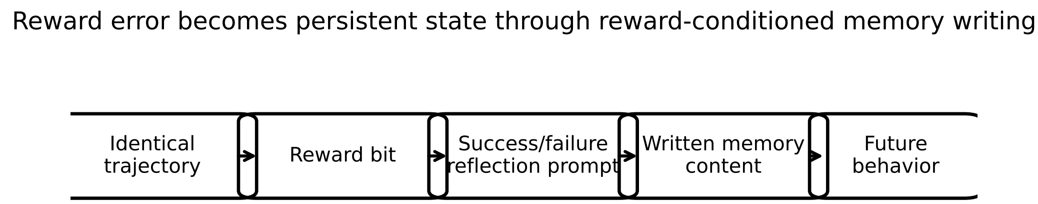

Figure 1: Reward-conditioned memory writing. A perturbation of the terminal reward bit changes the reflection objective for an identical trajectory. The resulting memory becomes persistent state and can affect future behavior.

complementary observation. Proxy reward errors can accumulate under repeated memory reuse and distort future learning value (Asadolahi et al., 2026).

We test the first link directly using released trajectories. For each sampled trajectory, the action history remains identical. The task remains identical. The writer model remains identical. We change only the reward label that selects the ReasoningBank reflection mode. Four complete paired interventions all produce different memory text. Their title sets also differ in every pair. Mean token-set Jaccard distance is 0.735.

The paired experiment establishes a write-channel mechanism. It shows that a reward-label perturbation can survive as a persistent memory perturbation even when the underlying trajectory is fixed. We then connect this mechanism to downstream variance with a matched memory-injection protocol. Paired memories are injected into the same future task under a fixed environment snapshot. Repeated runs measure early action divergence and terminal success dispersion.

Our contribution is a causal decomposition of reward noise in self-improving agents. Reward error first changes the memory-writing objective. The written memory then changes future context. Reuse can amplify this state perturbation across later episodes. This view turns evaluator reliability into a state-consistency requirement for systems that learn from persistent memory.

2 Reward Error as a Memory-Write Intervention

Let an episode produce task x with trajectory τ. The evaluator label is $r \in \{0, 1\}$. A reward-conditioned memory writer constructs

$$M = G(\tau, r). \tag{1}$$

The released ReasoningBank code instantiates this dependence directly. It maps score one to a success writer. Score zero selects a failure writer. The success prompt asks for the reason the trajectory succeeded. The failure prompt asks for the reason it failed and requests a corrective strategy.

This branch occurs before the memory is persisted. The reward label therefore changes agent state even when the trajectory is unchanged. Future retrieval uses the historical task description as the retrieval key. The reward label does not modify that key. It changes the memory content returned after retrieval. This structure isolates a clean causal channel from reward to future context.

For a future task x', let A denote the action trajectory under retrieved memory M. Future behavior follows

$$A = H(x', M, S), \tag{2}$$

where S contains the remaining fixed agent state. A proxy reward error ϵ changes r and induces a memory perturbation

$$\delta_M(\tau) = G(\tau, r \oplus \epsilon) - G(\tau, r). \tag{3}$$

Table 1: Paired memory divergence under the reward-label intervention.

Task ID	Original outcome	Token Jaccard distance
21	failure	0.691
22	failure	0.708
23	success	0.770
25	success	0.770

The first causal question is whether δ_M is nonzero under a fixed trajectory. The second question is whether this persistent perturbation changes future action distributions.

2.1 Variance amplification

Suppose repeated self-improvement runs experience small reward errors at different episodes. Each error can write a different memory. Later retrieval exposes future tasks to different persistent contexts. The resulting action distribution acquires an additional variance component from memory state:

$$\mathrm{Var}(Y) = \mathbb{E}_M[\mathrm{Var}(Y \mid M)] + \mathrm{Var}_M(\mathbb{E}[Y \mid M]). \tag{4}$$

The second term is the memory-state variance induced by the learning process. Reward-conditioned memory writing gives proxy reward noise a direct route into this term.

This mechanism is especially important for external-memory agents because the state update happens immediately. The system does not need parameter training for reward noise to persist. One mislabeled episode can alter future context as soon as the new memory is retrieved.

3 F0: Does the Reward Bit Change Written Memory?

We test the write channel with a paired intervention over released Salesforce trajectories. The source parquet contains task prompts and full action histories from the published WebArena runs. We select six trajectories through a deterministic rule that balances original outcomes. Three trajectories originally receive benchmark success. Three originally receive benchmark failure.

For every trajectory, we construct two ReasoningBank memory prompts. The first uses the released success-reflection system prompt. The second uses the released failure-reflection system prompt. The task text and action history are copied byte-for-byte into both conditions. DeepSeek-V4-Flash writes the paired memories at temperature zero. Provider responses are archived before analysis through content-addressed raw receipts.

Four trajectory pairs return complete memory outputs in both intervention arms. Every complete pair changes the normalized memory content. Every complete pair also changes the extracted memory-item title set. Token-set Jaccard distance ranges from 0.691 to 0.770. The mean distance is 0.735. Table 1 reports the pair-level values.

The intervention changes more than surface phrasing. For task 21, the success writer produces guidance centered on opening the review section and preserving direct quotes. The failure writer emphasizes exhaustive pagination and extraction verification. Task 23 shows a similar causal split. The success writer highlights evidence collection across pages. The failure writer focuses on pagination state and broader synonym coverage. The trajectory supplied to both writers is identical in each pair.

The result survives the F0 falsifier. Reward-conditioned memory writing is a real state channel in the released implementation. The effect appears in both original-outcome strata, which rules out a simple explanation based on selecting only trajectories that already failed.

This establishes the first stage of the paper mechanism:

$$\text{reward label} \longrightarrow \text{memory content}. \tag{5}$$

The next experiment tests the second stage by injecting the paired memories into matched future tasks.

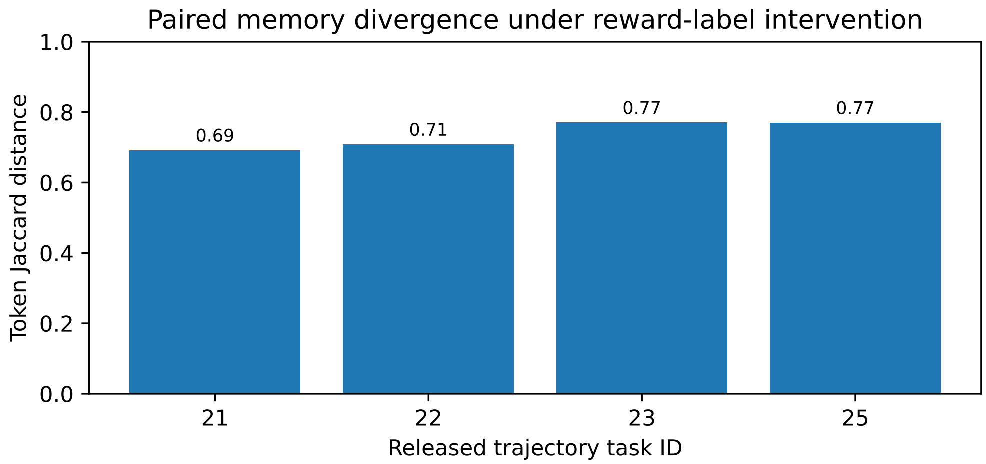

Figure 2: The same released trajectory produces different persistent memories when the reward-conditioned reflection mode is flipped.

4 Downstream Variance Protocol

F0 establishes that the reward bit changes persistent memory. The paper's second experiment measures how that state perturbation changes future behavior.

4.1 Matched future-task injection

Each complete F0 pair defines two memory conditions for the same historical task. We identify future tasks whose retrieval key matches that task family. The future query is fixed across conditions. The browser state starts from the same snapshot. The agent model uses the same decoding configuration. One condition receives the success-label memory. The paired condition receives the failure-label memory.

For each future task, repeated rollouts estimate two quantities. The first quantity is terminal success probability. The second quantity is early action divergence. We define the first-divergence step as the earliest decision at which the two memory conditions select different actions under matched observations.

The causal contrast for future success is

$$\Delta_Y(x') = \mathbb{E}[Y \mid do(M = M_F), x'] - \mathbb{E}[Y \mid do(M = M_S), x']. \tag{6}$$

Here M_F is the memory written under the failure label and M_S is the paired success-label memory from the identical source trajectory. A negative value means the failure-label interpretation reduces future success.

4.2 Run-level variance

We then construct memory banks under controlled reward corruption rates. The clean condition uses ground-truth reward. Corrupted conditions flip a frozen subset of episode labels before memory writing. Every run sees the same task order and the same environment reset schedule. The corruption mask is the only learning-signal intervention.

For each corruption level, we measure the dispersion of downstream success across repeated self-improvement runs. Equation 4 predicts that reward corruption increases the between-memory component when reward-induced memory differences affect future decisions. We also measure memory-bank divergence by comparing retrieved memory content for the same future query.

4.3 Mechanism prediction

The mechanism makes a directional prediction. Greater reward corruption should increase memory-state diversity. Future trajectories should diverge earlier when they retrieve reward-sensitive memories. Run-level success variance should rise when those divergences affect task completion.

This protocol connects the local write-channel result to the global variance phenomenon reported in self-improvement experiments (Ye et al., 2026). It also separates reward noise from benchmark order effects because the task order remains fixed during this intervention.

5 Related Work

Self-improving memory. ReasoningBank converts trajectories into persistent reasoning memories and reuses them on future tasks (Ouyang et al., 2026). Agent Workflow Memory learns reusable procedures from interaction histories (Wang et al., 2024). These systems demonstrate that external memory can produce strong test-time improvement. Our analysis focuses on the learning signal that decides how an episode is represented inside that state.

Fragility across self-improvement runs. The recent fragility study measures substantial run variance under online memory construction (Ye et al., 2026). Its design uses ground-truth reward to remove evaluator noise from the main analysis. We study the excluded causal channel directly and show that the reward label changes persistent memory under a fixed trajectory.

Reward inflation in memory systems. Memory Reward Inflation shows that imperfect proxy reward can inflate the effective value of stored memories and propagate through later retrieval (Asadolahi et al., 2026). Our F0 identifies an earlier point in the same learning loop. Reward error can change the content being stored before subsequent memory selection or utility amplification occurs.

Relation to label noise. Supervised learning studies label noise as corruption of a training target. External-memory self-improvement exposes a different update path. A single terminal label can choose a different natural-language reflection objective and immediately write a new persistent context. The state perturbation becomes available to the next task without gradient training.

6 Limitations

The F0 intervention requests six paired memories. Four pairs return complete outputs in both reward conditions. Two failure-label provider responses terminate as incomplete before assistant text is produced. Their provider response identifiers are archived and GET-only recovery confirms that no text is available. These two cases remain support debt. They do not change the observed 4/4 content-divergence result among complete pairs.

F0 establishes the reward-to-memory write channel. It does not yet measure the full reward-to-future-outcome effect. Matched future-task injection and run-level corruption experiments remain experiment debt. The manuscript therefore treats downstream variance amplification as an active causal hypothesis grounded by the observed write-channel mechanism.

The paired intervention uses DeepSeek-V4-Flash as the memory writer. The released ReasoningBank implementation can use other writer models. Replication across memory-writer families remains experiment debt for a model-independent effect-size claim.

The released trajectories used for F0 come from one WebArena Shopping run. They include both original-success and original-failure episodes, which supports the mechanism across outcome strata. Broader task-family replication remains experiment debt.

Token-set Jaccard distance measures content divergence at a coarse lexical level. Title-set changes supply an additional structural measure. A semantic representation analysis can further quantify whether reward labels move memories toward distinct strategy clusters.

7 Conclusion

Reward-conditioned memory writing creates a direct path from evaluator error to persistent agent state. In the released ReasoningBank implementation, one reward bit selects a different reflection objective for the same trajectory. Our paired F0 intervention observes memory-content changes in every complete pair and measures mean token-set Jaccard distance of 0.735.

This mechanism explains how reward noise can survive beyond one episode without parameter training. A corrupted reward can change the memory that future tasks retrieve. Repeated memory reuse can then turn a local evaluation error into run-level behavioral variance. The resulting design principle is simple: reward reliability must be treated as state reliability whenever an agent writes persistent memory from outcomes.

References

Mohammad Asadolahi, Amir Amini, Samira Talebi, Amirfarhad Farhadi, and Azadeh Zamanifar. Memory reward inflation in self-improving llm agents, 2026.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, 2026.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024.

Qinyuan Ye, Yu Li, Yada Pruksachatkun, Jiaxin Zhang, and Chien-Sheng Wu. On the fragility of self-improving agents: Variance, task order, and underspecification, 2026.

A F0 Reproduction Details

A.1 Released trajectory sample

The F0 sample uses six task IDs from the released AWM shuffle-seed-42 Shopping parquet: 21 through 25, plus 47. The deterministic sampler selects the lowest task IDs with parseable trajectory records within each original-outcome stratum. Original failures are 21 through 22 plus 24. Original successes are 23 plus 25 and 47.

A.2 Memory construction intervention

Each released trajectory is summarized into an action trace. The same trace is inserted into two prompts. One prompt contains the released ReasoningBank success-reflection system instruction. The paired prompt contains the released failure-reflection system instruction. The task prompt stays fixed. The writer model stays fixed. Temperature stays at zero. The raw provider output is archived before metric computation.

A.3 Pair-level F0 results

Table 2: Complete paired interventions.

Task	Original outcome	Content changed	Token distance
21	failure	yes	0.691489
22	failure	yes	0.708108
23	success	yes	0.769953
25	success	yes	0.769608

The extracted memory-item title set changes for every complete pair. The mean token-set Jaccard distance is 0.734789. Provider failures for task 24 and task 47 are stored as support failures with zero scientific authority.

A.4 Downstream experiment contract

Future-task experiments reuse paired memory outputs from F0. The future query remains fixed across the pair. The benchmark service state is restored from the same snapshot. The model configuration is held constant. Repeated rollouts record terminal success and the first action-divergence step.